

ID3802 – Final Report

Project ID	P191
Date	17-05-2026
Project Title	Necessarily Rank-Maximal Matching
Students	Madhav P Nair (112301020), Devapriya Pradeep (112301006)
Mentor	Dr.Pratik Ghosal

1 Introduction

1.1 Project overview

The Necessarily Rank-Maximal matching is a niche but promising area in graph theory. Here we address the problem of resource allocation and try to optimize it using an optimality criterion called Rank maximality. We deal with the cases where agents have preferences over some objects but not vice versa.

Consider a set of students and a set of projects where each student has a preference list in which each project gets a specific rank. We work with the online variant of the problem where we start with an empty preference table and elicit the preferences of the students over the projects using a **query model** and return an optimal matching once sufficient information is retrieved without any further queries.

Jannik Peters[3] put forward an open question in his paper to improve the upper bound of the online elicitation algorithm for rank maximal matching based on the set compare model which is $O(n)$ (trivial) where n is the number of agents/objects. We addressed this open challenge and modified the set compare model without deviating from its basic principles, but retrieving more information from an agent in each query. We propose the **Rank-Aware set-compare model** based on this.

We came up with an **original algorithm** for the online elicitation of rank maximal matching based on the RASC model.

1.2 Approach

An optimal algorithm is a hypothetical algorithm that has complete knowledge of the entire input in advance and uses this knowledge to determine the minimum number of queries needed to compute an NRM matching.

We say that an online elicitation algorithm is α -competitive or has competitive ratio α if the number of queries it requires to ask in the worst-case across all possible instances is at most $\alpha \cdot \text{OPT}$ where OPT is the number of queries asked by the optimal algorithm.

We focus on the analysis and proof of the algorithm presented. We prove (or disprove) that the algorithm has a constant competitive ratio, which is an improvement from the linear competitive ratio of the set compare query model.

Moreover, an NRMM simulator using the original algorithm based on the RASC model is also developed and deployed. This visualizes every step of the algorithm and produces the NRM matching for agent – object systems of any size and preference distributions.

2 Literature Survey

2.1 Peters’ Analysis

We read the work of Peters on Online Elicitation of Optimal Matchings [3]. Our original algorithm is essentially inspired from the open question proposed by Peters in his paper on Online Elicitation of Necessarily Optimal Matchings. It is very fundamental to comprehend his algorithms, the corresponding optimal offline algorithms and the techniques he employed to come out with competitive ratios(α) for moving forward. He gives algorithms for online elicitation of necessarily rank-maximal matchings based on both next-best query model and hybrid query model. Our primary focus was on the online algorithm he proposed based on the next-best query model, the proof of correctness, and competitive ratio analysis for the same.

The algorithm is followed by the correctness proof of the algorithm. The author introduces a couple of terms and related constraints which are helpful in the upcoming α - analysis. After setting up the plot, the author goes for deriving an upper bound of 3 for α . This is followed by the rigorous analysis for the tight upper bound of $3/2$.

2.2 Plot - Lemma 5

Lemma 5 of [3] acts as a vital step to show a constant bound for the competitive ratio for the algorithm proposed by Jannik Peters. This ensures that the competitive-ratio will have a constant bound of 3 and leaves room for further improvement.

Lemma 2.1. *Given a set of agents A , objects O and a partial preference profile $\succ$ in the next-best query model and an NRM M for the problem, $|\text{rev}(a)| \geq r_a - 1$ for all agents a matched to a revealed preference profile by M . Here r_a is the iteration at which agent a is no longer even.*

Proof. We assume to the contrary that there exists an agent a with $M(a) \in \text{rev}(a)$ and $|\text{rev}(a)| < r_a - 1$. We know that $a \in \mathcal{E}_{r_a-1}$ by the definition of r_a , which means that there is an even length alternating path from a to a free vertex a_k . Since a_k is free, the pair matched with a_k in M must have a rank higher than $r_a - 1$, i.e., $\text{rank}(a_k, M(a_k)) > r_a - 1$.

Assume that there exists an a_i with $i > 0$ such that $\text{rank}(a_i, M(a_{i-1})) > \text{rank}(a_{i-1}, M(a_{i-1}))$ for a_i in the even alternating path between a and a_k . Since a is even, all a_i in the path are also even in the $(r_a - 1)$ st iteration. This means that the pair matched to a_{i-1} becomes odd at the iteration at which it is matched, i.e., $M(a_{i-1}) \in \mathcal{O}_{\text{rank}(a_{i-1}, M(a_{i-1}))}$. Since the algorithm treats odd objects as inactive, they do not participate in any further path augmentations. Therefore, $M(a_{i-1})$ cannot be matched with an agent that is not a_i in G_{r_a-1} . That is, the edge $(a_i, M(a_{i-1})) \notin G_{r_a-1}$. This contradicts our assumption, and therefore, $\text{rank}(a_i, M(a_{i-1})) \leq \text{rank}(a_{i-1}, M(a_{i-1}))$ for all a_i in the alternating path with $i > 1$. Furthermore, $\succ$ can be extended so that $\text{rank}(a_1, M(a_k)) \leq r_a - 1$.

Taking into account the even length alternating path starting from a and ending at a_k , we can augment M to get a new matching M' which has $\text{rank}(a_i, M'(a_i)) = \text{rank}(a_i, M(a_{i-1})) \leq \text{rank}(a_{i-1}, M(a_{i-1}))$ for all a_i , $i > 1$ and $\text{rank}(a_1, M(a_k)) \leq r_a - 1$ while $\text{rank}(a_k, M(a_k)) > r_a - 1$ previously. Thus, M' is strictly a better rank-maximum matching compared to M . This contradicts our assumption that M is NRM and therefore $|\text{rev}(a)| \geq r_a - 1$ for all $a \in A$. $\square$

That is, atleast $|r_a - 1|$ queries are needed for each agent a by the optimal algorithm to determine the NRM matching.

2.3 Upper bound

The author adopts a classic strategy in competitive analysis. This approach starts by kind of glorifying the OPT to create a safety margin (constant-competitive) and later applies the real structural constraints and thus grounds the adversary. It starts with some assumptions for this loose analysis

- The optimal algorithm should match at least $n - 1$ agents to revealed posts
- If an agent is matched to a revealed post, $r_a - 1$ preferences must be revealed for this particular agent
- In our algorithm, each agent has to reveal r_a posts

So,

$$ALG \leq \Sigma(r_a) + n - 1$$

and

$$OPT \geq \Sigma_{a \in A'}(\max(r_a - 1, 1))$$

Thus we get

$$\begin{aligned} \frac{ALG}{OPT} &\leq \frac{\sum_{a \in A'}(r_a) + n - 1}{\sum_{a \in A'}(\max(r_a - 1, 1))} \leq \frac{\sum_{a \in A'}(r_a + 1)}{\sum_{a \in A'}(\max(r_a - 1, 1))} \\ &\leq \frac{\sum_{a \in A'}(r_a + 1)}{\sum_{a \in A'}(\max(r_a - 1, 1))} \leq \max_{a \in A'} \left(\frac{r_a + 1}{\max(r_a - 1, 1)} \right) \leq 3 \end{aligned}$$

2.4 Tight analysis for 3/2

Here we employ more rigorous structural comparisons and derive the tight bound of 3/2 for α . Instead of isolating agents, the method accounts for **resource conflicts** by **grouping agents** who compete for the same house which forces the OPT also to work harder to resolve conflicts.

2.4.1 The unrevealed case

Suppose agent a' matches to an unrevealed house. Then this leftover house should be the last preference for all other agents or none of the houses in the revealed matchings are revealed by a' . Otherwise they could swap houses to make a better matching.

Here, OPT has to query every preference of each agent who match to a revealed house to make sure that the leftover house is their worst preference and ALG seeks every preference of all agents.

$$\text{OPT} = (n-1)^2$$

$$\text{ALG} = n(n-1)$$

$$\frac{\text{ALG}}{\text{OPT}} = \frac{n}{n-1}$$

which is at most $3/2$ for $n \geq 2$

2.4.2 Grouping by first choice

The author groups agents by the first-choice house to capture the resource contention. A_h is defined as the set of all agents whose first choice is house h . OPT should ask at least $2|A_h|-1$ queries to this group which can be interpreted as two queries each to all the losers who have better fallback option as the second choice and one query to the winner. $\text{OPT}(A_h)$ is the minimum number of queries the optimal algorithm must ask the specific group of agents who rank house h as their first choice.

$$\text{OPT}(A_h) \geq \max(\sum_{a \in A_h} (\max(r_a - 1, 1)), 2|A_h| - 1)$$

$$= \max(\text{individual necessity}, \text{group conflict})$$

$$\text{OPT} = \sum_{h \in H} \text{OPT}(A_h)$$

$$\text{ALG} = \sum_{h \in H} \sum_{a \in A_h} r_a$$

The author applies the powerful **mediant rule** ie.,

$$\frac{\sum a_i}{\sum b_i} \leq \max(\frac{a_i}{b_i})$$

Thus the upper bound reduces to

$$\frac{\text{OPT}}{\text{ALG}} \leq \max_{h \in H} \frac{\sum_{a \in A_h} r_a}{\text{OPT}(A_h)}$$

Now we have two cases :

1. **High query case:** $\sum_{a \in A_h} r_a \geq 3|A_h|$

$$\frac{\text{OPT}}{\text{ALG}} \leq \max_{h \in H} \frac{\sum_{a \in A_h} r_a}{\text{OPT}(A_h)} \leq \max_{h \in H} \frac{\sum_{a \in A_h} r_a}{\max(\sum_{a \in A_h} \max(r_a - 1, 1), 2|A_h| - 1)} \leq \max_{h \in H} \frac{\sum_{a \in A_h} r_a}{\sum_{a \in A_h} \max(r_a - 1, 1)}$$

$$\frac{\text{OPT}}{\text{ALG}} \leq \max_{h \in H} \frac{\sum_{a \in A_h} r_a}{r_a - |A_h|} \leq \frac{\sum_{a \in A_h} r_a}{r_a - \frac{1}{3} \sum_{a \in A_h} r_a} = \frac{\sum_{a \in A_h} r_a}{\frac{2}{3} \sum_{a \in A_h} r_a} = \frac{3}{2}.$$

2. **Low query case:** $\sum_{a \in A_h} r_a < 3|A_h|$

There exists at least one $a \in A_h$ with $r_a = 2$ since $\sum_{a \in A_h} r_a < 3|A_h|$.

Let $A_h^2 = \{a \in A_h \mid r_a = 2\}$.

(a) $|A_h^2| \geq 2$

$$\begin{aligned} \frac{\sum_{a \in A_h} r_a}{\text{OPT}(A_h)} &= \frac{\sum_{a \in A_h \setminus A_h^2} r_a + 2|A_h^2|}{\text{OPT}(A_h \setminus A_h^2) + \text{OPT}(A_h^2)} \\ &\leq \max \left(\frac{\sum_{a \in A_h \setminus A_h^2} r_a}{\sum_{a \in A_h \setminus A_h^2} r_a - |A_h \setminus A_h^2|}, \frac{2|A_h^2|}{2|A_h^2| - 1} \right) \leq \frac{3}{2}. \end{aligned}$$

Since $\sum_{a \in A_h \setminus A_h^2} r_a \geq 3|A_h \setminus A_h^2|$ and it had already been proven that for such a case, $\frac{ALG}{OPT} \leq \frac{3}{2}$, the first argument for max is bounded by $\frac{3}{2}$.

$$\frac{2|A_h^2|}{2|A_h^2| - 1} \leq \frac{3}{2} \quad \text{for all } A_h \text{ such that } |A_h| \geq 2.$$

Therefore, we get an upper bound of $\frac{3}{2}$.

(b) $|A_h^2| = 1$

There is exactly one $a \in A_h$ with $r_a = 2$. So a is O or U in the 2nd iteration. Since all other agents are E in the 2nd iteration and they all have edges to h , h can't be even in the second iteration since EE edges are not allowed. So (a, h) Since h is unavailable, and a gets out of E in the 2nd iteration, a must be matched to its 2nd preference. So, OPT must ask a 2 queries.

$$\frac{ALG}{OPT} \leq \frac{\sum_{a \in A_h} r_a}{\max(\sum_{a \in A_h} \max(r_a - 1, 1), 2|A_h| - 1)} \leq \frac{3|A_h| - 1}{2|A_h| - 1} \approx \frac{3|A_h| - 1}{2|A_h|} = \frac{3 - \frac{1}{|A_h|}}{2} \leq \frac{3}{2}.$$

Thus, the algorithm has a competitive ratio of $\frac{3}{2}$.

3 Analysis of RASC Algorithm

3.1 Approach

Here, we employ an agent-specific approach where we consider an agent a and compare the query cost of OPT and ALG for the same agent. Further, we try to impose a lower bound constraint on OPT w.r.t the ALG on the number of queries to a . This is done by proving that OPT fails to produce an NRM matching when it violates the proposed constraint. This proof by contradiction is done by strategically manipulating the preferences which the OPT hesitates to reveal and claims that it has produced some NRM matching.

3.2 Analysis

Let us look into some definitions and lemmas which are helpful for the competitive ratio analysis

Definition 3.1. [1] Let $G = (A \cup P, E)$ be a bipartite graph with ranks on the edges belonging to $\{1, 2, \dots, r\}$. Suppose that $a \in A, p \in P$ and (a, p) does not belong to E . Let $H = (A \cup P, E \cup \{(a, p)\})$. We denote a critical rank of (a, p) in H as follows.

Lemma 3.1. [1] Let G, H and (a, p) be as in Definition 4. Then the critical rank of (a, p) is c if and only if

- For every $1 \leq i < c$, the edge (a, p) belongs to every rank-maximal matching of H_i , in which (a, p) has rank i , and
- For every $c < i \leq r$, the edge (a, p) does not belong to any rank-maximal matching of H_i , in which (a, p) has rank i , and
- There exists a rank-maximal matching M of H_c , in which (a, p) has rank c such that (a, p) is not contained in M .

Lemma 3.2. If we swap positions of two posts p_i and p_j both belonging to r_a , where $\text{rank}(a, p_i) < \text{rank}(p_j)$ in the preference list of a , then after the swap a will be matched to p_j in every RMM. Here, r_a denotes the posts that are revealed by the algorithm for an agent a .

Proof. Since p_j gets queried after p_i is processed, critical rank of $(a, p_j) \geq \text{rank}(a, p_i) + 1$.

On swapping, p_j will be at the position $\text{rank}(a, p_i) \leq \text{critical rank of } (a, p_j) - 1$ i.e., $\text{rank}(a, p_i) < \text{critical rank of } (a, p_j)$. Therefore, by lemma 2, p_j will be present in every rank maximal matching. $\square$

Lemma 3.3. The optimal algorithm must match every agent except at most 1 to a revealed post.

Proof. [2] The optimal algorithm returns a matching M which is an NRM. We know that all NRMs are also NPOs (Necessarily Pareto Optimal Matching). Now, consider the case where optimal algorithm is matched to 2 agents which are unrevealed. Let them be a_1 and a_2 matched to unrevealed objects o_1 and o_2 respectively. Since they are unrevealed, we can make a completion of the preference lists of a_1 and a_2 so that a_1 places o_2 before o_1 and a_2 places o_1 before o_2 in their respective preference lists.

Consider a matching M' where all agents a' in $A' = A \setminus \{a_1, a_2\}$ are matched to $M(a')$ and a_1 and a_2 are matched to o_2 and o_1 respectively. This matching makes a_1 and a_2 better off and since no one else is worse off, M' pareto dominates M , contradicting the fact that M is an NPO. Since M is not an NPO, it is not an NRM either. Therefore, by contradiction we achieve that the optimal algorithm must match at least $n - 1$ agents to their revealed preferences. $\square$

Lemma 3.4. *Optimal must reveal at least $q_a - 2$ objects for an agent a that is revealed, where q_a denotes the number of agents revealed by the algorithm for a .*

Proof. Let us consider a case where the optimal has revealed only $q_a - 3$ objects for an agent a .

Consider a general case where o_i is queried at iteration $i' \leq i$, o_j at iteration $j' \leq j$ and o_k at iteration $k' \leq k$. We divide this into various cases depending on the object a is matched to in the NRM. We utilize lemma 3 in these cases.

- If a is matched to a revealed object, critical rank of $(a, o_i) < \text{rank}(a, o_j)$ so o_i and o_j can be swapped. By lemma 3, o_j must be matched in every rank-maximal matching and therefore the NRM of the preference profile.
- If a is matched to o_i , critical rank of $(a, o_i) < \text{rank}(a, o_j)$ so o_i and o_j can be swapped to make o_j matched to a in the NRM.
- If a is matched to o_j , critical rank of $(a, o_j) < \text{rank}(a, o_k)$ so o_k and o_j can be swapped to make o_k matched to a in the NRM.
- If a is matched to o_k , critical rank of $(a, o_i) < \text{rank}(a, o_j)$ so o_i and o_j can be swapped to make o_j matched to a in the NRM.

In all cases, we prove that the agent matched to a in the NRM is different to that returned by the algorithm, thus proving that optimal is wrong in such a scenario.

This contradicts our assumption that optimal algorithm can work with $q_a - 3$ revealed agents. □

The following is a visualization of the above discussed proof.

Here, we try to show that the OPT has to do at least ALG - 2 queries to ensure that the matching produced by it will remain rank-maximal irrespective of how the unrevealed preferences are filled.

We try to produce an instance where the OPT ends up asking 3 queries less than the ALG and claims it has got a rank-maximal matching. We do a case-by-case analysis of the above scenario, disprove the claim of OPT and thus prove the lemma by **contradiction**.

Consider an agent a . The following illustrates the preferences of a revealed by the RASC-based ALG.

Agent	$r = 1$	$r = 2$	$r = 3$	$r = 4$	$r = 5$	$r = 6$	$r = 7$	$r = 8$	$\dots$	$r = k$
a	o_1	$\times$	$\times$	$\odot_4$	$\times$	$\odot_6$	o_7	$\odot_8$	$\dots$	o_k

o_i denotes the i^{th} choice of a and $r = i$ means i^{th} rank.

$k \leq n$ (total number of agents or objects) and the agent a is Even till k^{th} iteration (since she is queried till k^{th} iteration).

Recall how ALG works. Some of the preferences of a are not queried by the ALG since the posts are already Odd or Unreachable by the time of that iteration and those skips are marked as **X**. Moreover, the ALG stores the next best choice of a from the available posts in memory in this iteration.

W.L.O.G, we say that in the partial preference list of a revealed by OPT, three choices - o_4 , o_6 and o_8 are not revealed (marked in circles) and still OPT claims that it has produced a rank-maximal matching having (a, o) , ie., agent a is matched to some object o .

We break down this choice of o into different cases and find a better matching in each case. We leverage the fact that we are free to **manipulate the positions which are not revealed by OPT** and can permute as we wish to prove OPT wrong.

We make use of the concept of **critical rank**[1] (see Definition 3.1 and Lemma 3.1). Critical rank c of (a, p) is defined as the first iteration where either the agent a or the object o becomes Odd or Unreachable. For every $1 \leq i < c$, the edge (a, p) belongs to every rank-maximal matching, in which (a, p) has rank i

The object o can be classified into five categories comparing its rank with the rank of unrevealed(OPT) objects. In each case, **we promote an object o^* unrevealed by OPT in the preference list to a position which ensures Even-ness for the object, so that $\text{rank}(a, p)$ is strictly less than its critical rank. This ensures (a, o^*) is matched in every rank-maximal matching.** We place the promoted object on or before the position where it was queried from. This object has to be Even (available) in this region.

1. $rank(a, o) < rank(a, o_4)$: swap the positions of o_4 and o_6 . Every rank-maximal matching should contain (a, o_6) and so we discard the matching given by OPT.
2. $o = o_4$: swap the positions of o_4 and o_6 . Every rank-maximal matching should contain (a, o_6) . The matching given by OPT no more remains rank-maximal.
3. $o = o_6$: swap the positions of o_6 and o_8 . We promote o_8 and match it with a . Every rank-maximal matching should contain (a, o_8) and thus we discard the matching given by OPT. **Here, it makes no sense to promote and use o_6 to match with a since o_6 is the one given by OPT. This is the tighter case which forces us to have three unrevealed objects to efficiently manipulate the list.**
4. $o = o_8$: swap the positions of o_4 and o_6 . Every rank-maximal matching should contain (a, o_6) and thus we discard the matching given by OPT.
5. $rank(a, o) > rank(a, o_8)$: we can promote either o_6 or o_8 to get a better match for a which is obviously not o .

Here, in each case we could prove OPT is wrong. So, OPT cannot afford asking three or more queries less than the ALG and hence the above stands proved.

This lemma concludes that in RASC-based NRM elicitation, the OPT has to perform at least ALG - 2 queries to each agent matched to a revealed post.

$$OPT \geq ALG - 2$$

Lemma 3.5. *The algorithm is a 4-competitive algorithm for eliciting a rank-maximal matching in the RASC query model.*

Proof. We calculate the competitive ratio:

$$\alpha = \frac{ALG}{OPT} \leq \frac{\sum_{a \in A'} q_a + n - 1}{\sum_{a \in A'} \max(1, q_a - 2)} = \frac{\sum_{a \in A'} (q_a + 1)}{\sum_{a \in A'} \max(1, q_a - 2)}$$

Here, A' denotes the set of all agents that are matched to revealed objects.

All agents in A' are asked q_a queries by the algorithm as per the definition of q_a . The last agent which might or might not be matched to an unrevealed object will be asked a maximum of $n - 1$ queries.

Optimum must ask atleast $q_a - 2$ queries for an agent a by lemma 3.4. However, in case of $q_a \leq 2$, optimal can not finalize with 0 queries asked, hence we assign 1 query to such a case for the optimal algorithm.

α is at most 4 at $q_a = 3$. Thus, we conclude that our algorithm gives an upper bound of 4 on the competitive ratio. $\square$

4 Implementation : RASC based NRMM Simulator

We developed and deployed a simulator for the elicitation of rank-maximal matchings using the proposed algorithm which works on the RASC model. This web app is deployed on render and is live at the following link.

RASC-based NRMM Simulator

4.1 Tech Stack

- FastAPI and Python for the backend
- Javascript for the frontend
- In-memory dictionaries for state management

4.2 Backend

- Used the **FastAPI** Python framework to develop the backend of the simulator
- Used the **NetworkX** library for basic graph operations such as computing maximum matchings
- Developed a Python library consisting of two modules: **edmonds_gallai** and **graph_utils** to implement the algorithm
- **Websocket** facilitates bi-directional (queries to agents and responses from agents) real-time communication
 - We implemented an asynchronous function that takes the number of agent-object pairs as input and returns a Necessarily Rank Maximal matching through an interactive process, by continuously querying users for their preference lists.
 - A websocket endpoint `/ws/nrm` was developed which utilizes the function mentioned above and maintains a persistent communication with the user.
- The backend is made robust by adding **runtime error handling mechanisms** to spot invalid inputs, inconsistencies in user input and catch if the agents cheat or try to manipulate their preferences. The major checks include:
 - Incomplete user input such as a missing object or object rank.
 - Inconsistent preferences such as input of a lower-ranked object after a higher ranked object or failing to provide a first choice in the first algorithm iteration. *This is done by tracking the latest rank revealed by all agents*
 - Input of a preference rank that exceeds the number of agent-object pairs that exist.
 - Making an **illegal jump** in the preference list by providing a less preferred object when a better one is available. *This is done by tracking of the number of inactive agents*

4.3 Features

- The simulator queries each agent and retrieves responses in an online fashion
- The application scales to support any number of agents and objects, and computes the NRM matching for arbitrary preference distributions
- It visualizes each step of each iteration of the algorithm
- Updates and renders the bipartite graph after each iteration
- Robust error handling catches inconsistencies in inputs and cheating
- Real-time logging of every event
 - Query
 - Response
 - Warnings for invalid responses
 - Storing future preferences
 - Skipping queries
 - Cache hit

- Edges added after each iteration
 - System logs related to connection status
- Shows the state of the in-memory dictionary which stores the future preferences to reduce query costs
- Produces the final NRM matching along with the query count and the signature of the matching

The complete project has been documented and uploaded to the following GitHub repository.

Necessarily-Rank-Maximal-Matching-GitHub

References

- [1] Pratik Ghosal and Katarzyna Paluch. Manipulation strategies for the rank maximal matching problem. October 2017.
- [2] Hadi Hosseini, Vijay Krishna Menon, Nisarg Shah, and Sujoy Sikdar. Necessarily optimal one-sided matchings. In *Proceedings of the 30th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 245–251. IJCAI, 2021.
- [3] Jannik Peters. Online elicitation of necessarily optimal matchings. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pages 5164–5172, 2022.